

FIAP

VISÃO COMPUTACIONAL APLICADA

Introdução à Visão Computacional e Preparação do Ambiente

AULA 1 • 120 MINUTOS • AULA PRÁTICA

Prof. Claudiney Sanches • Curso de Visão Computacional • FIAP

O que vamos fazer nestes 120 minutos

1	Boas-vindas, ementa e objetivos do curso	10 min
2	Fundamentos: Processamento de Imagem x Visão Computacional	20 min
3	IA, Aprendizado de Máquina e Aprendizado Profundo aplicados à VC	20 min
4	Aplicações reais de Visão Computacional	15 min
5	Intervalo	10 min
6	Preparando o ambiente prático (navegador + editor + libs)	15 min
7	Mão na massa: primeiro programa com OpenCV.js	20 min
8	Desafio, recapitulação e próxima aula	10 min

Visão Computacional Aplicada

A disciplina capacita os alunos com conhecimentos teóricos e práticos sobre técnicas de visão computacional e suas aplicações em sistemas de software. Ao longo do curso, trabalharemos com algoritmos de processamento de imagem, reconhecimento de padrões e análise visual, desenvolvendo projetos que aplicam essas tecnologias em cenários do mundo real.

F

Fundamentos

Compreender os fundamentos da Visão Computacional e suas principais técnicas.

P

Prática

Aplicar algoritmos de VC em projetos práticos, com código real no navegador.

A

Análise visual

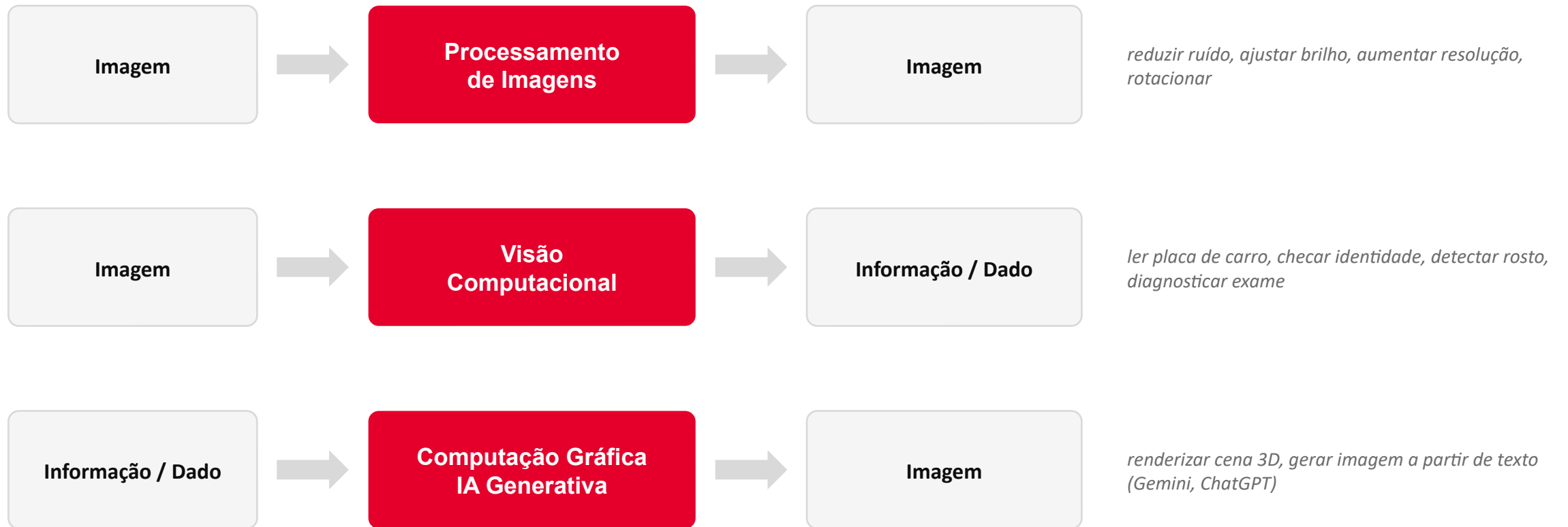
Desenvolver habilidades em processamento de imagem e análise de dados visuais.

A

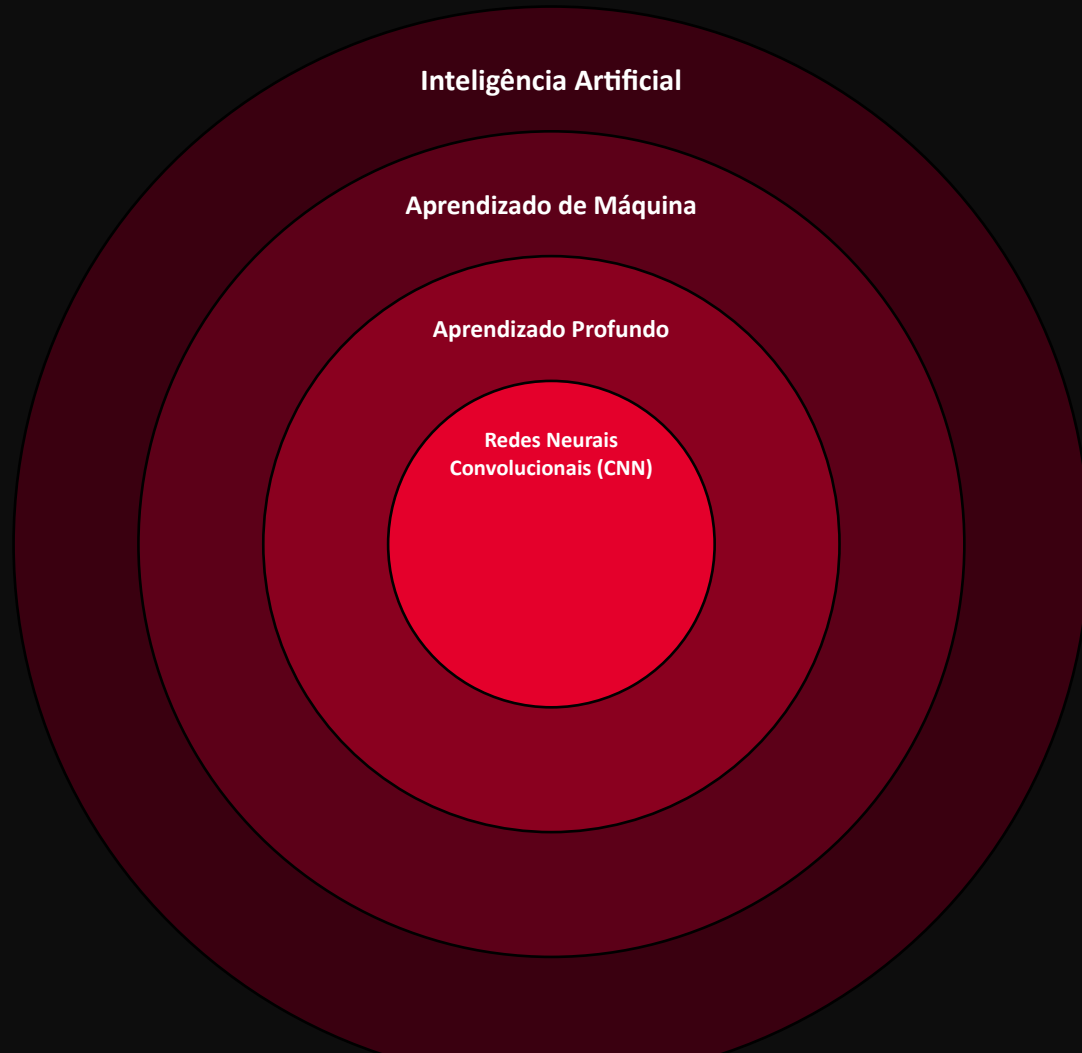
Aplicações de IA

Explorar VC em IA, automação e interação homem-máquina.

Três caminhos entre imagem e informação



IA, Aprendizado de Máquina e Aprendizado Profundo



1

Design direto: um humano escreve as regras (IF/ELSE).

2

Aprendizado de máquina: o computador aprende regras a partir de exemplos.

3

Aprendizado profundo (CNN): também aprende sozinho quais atributos observar na imagem.

Linha do tempo da Inteligência Artificial



“A atual onda de IA generativa tem pouco mais de uma década — mas o progresso foi extremamente rápido.”

Design direto x Aprendizado de máquina

Design direto

O programador escreve as regras manualmente (IF / ELSE).

```
// classificar rosto em M/F?  
if (???) {  
    // impossível descrever  
    // uma regra geral aqui!  
}
```

Impossível para tarefas visuais complexas

Aprendizado de máquina

O algoritmo aprende o modelo a partir de centenas de exemplos.

```
modelo.treinar(  
    centenasDeFotos("homem"),  
    centenasDeFotos("mulher")  
);  
modelo.classificar(novaFoto)
```

Erro de apenas ~1% em problemas assim

Onde a IA em visão computacional já está presente

C

Classificação de imagens

O sistema decide a que categoria uma imagem pertence, sem regras escritas manualmente.

Ex.: identificar espécie, diagnosticar radiografia, classificar grãos

D

Detecção de objetos

Localiza onde cada objeto está na imagem, indicando posição e escala.

Ex.: localizar produtos, placas, pessoas em uma cena

Onde a IA em visão computacional já está presente

S

Segmentação semântica

Classifica cada pixel da imagem, separando objeto de interesse e fundo.

Ex.: separar célula do fundo, remover background de uma foto

R

Reconhecimento facial

Compara um rosto capturado com um banco de referências para identificar a pessoa.

Ex.: login por rosto, controle de acesso, marcação automática de fotos

Onde a IA em visão computacional já está presente

P

Colorização automática

Aprende a converter fotos em preto-e-branco para colorido, a partir de milhares de exemplos.

Ex.: restauração de fotos e filmes antigos

Z

Super-resolução

Aumenta a resolução de uma imagem, reconstruindo detalhes plausíveis.

Ex.: melhorar nitidez de fotos antigas ou de baixa qualidade

Por que Visão Computacional no navegador?

1

Zero instalação pesada

Sem compilar C++, sem configurar IDE — só um navegador atualizado.

2

Multiplataforma

O mesmo código roda em Windows, macOS, Linux, tablets e celulares.

3

Fácil de compartilhar

Basta um link — ideal para portfólio e apresentação de projetos.

4

Performance real

OpenCV.js usa WebAssembly: quase a velocidade do C++ nativo.

Nosso stack tecnológico

HTML5

Estrutura da página e captura de mídia (câmera, arquivos)

CSS3

Layout e identidade visual das aplicações

JavaScript

Lógica da aplicação, com a API nativa do navegador (querySelector, addEventListener)

OpenCV.js

Algoritmos de visão computacional via WebAssembly

Conhecendo o OpenCV.js

OpenCV.js é a versão do OpenCV (a biblioteca de visão computacional mais usada do mundo, escrita em C++) compilada para WebAssembly. Isso significa que rodamos, dentro do navegador, praticamente os mesmos algoritmos usados em produção por empresas de todo o mundo — sem instalar nada.

cv.Mat estrutura de dados que representa uma imagem (matriz de pixels)

cv.imread / cv.imshow lê uma imagem de um `<img>/<canvas>` e a exibe em um `<canvas>`

cv['onRuntimeInitialized'] callback disparado quando o WebAssembly termina de carregar

```
index.html

<script src=
  "https://docs.opencv.org/
  4.x/opencv.js">
</script>

cv['onRuntimeInitialized'] =
  function () {
    console.log(
      "OpenCV.js pronto!");
  };
```

INTERVALO

Voltamos em 10 minutos para colocar a mão no código.

Preparando o ambiente de desenvolvimento

-  **Navegador atualizado**
Chrome, Edge ou Firefox nas versões mais recentes.
-  **Editor de código**
VS Code (recomendado) — gratuito, leve e com ótimas extensões.
-  **Extensão Live Server**
Recarrega a página automaticamente a cada alteração salva.
-  **Pasta do projeto**
Uma pasta com index.html, uma pasta css/ e uma pasta js/.
-  **Conexão à internet**
Para carregar o OpenCV.js via CDN (ou baixá-lo localmente).

Estrutura de pastas do projeto prático

```
projeto/  
  
aula01-hello-vision/  
├── index.html  
├── css/  
│   └── style.css  
├── js/  
│   └── script.js  
└── img/  
    └── flor.jpg
```

`index.html`

estrutura da página: título, canvas, controles

`css/style.css`

identidade visual FIAP da aula prática

`js/script.js`

lógica em JS puro + chamadas ao OpenCV.js

`img/`

imagens de exemplo para os exercícios

Importando o OpenCV.js

```
index.html - <head>

<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8" />
  <title>Aula 1 · Hello Vision</title>
  <link rel="stylesheet" href="css/style.css" />

  <!-- OpenCV.js, via CDN -->
  <script async src="https://docs.opencv.org/4.x/opencv.js"></script>
</head>
```

Hello Vision — estrutura HTML

```
index.html — <body>

<body>
  <h1>Aula 1 · Hello Vision</h1>

  <input type="file" id="inputImagem" accept="image/*" />
  <button id="btnCinza">Converter p/ cinza</button>

  <canvas id="canvasOriginal"></canvas>
  <canvas id="canvasSaida"></canvas>

  <script src="js/script.js"></script>
</body>
```

Hello Vision — JavaScript + OpenCV.js

```
js/script.js

// só usamos o OpenCV depois do WASM carregar
cv['onRuntimeInitialized'] = function () {
  const inputImagem = document.querySelector('#inputImagem');
  const btnCinza = document.querySelector('#btnCinza');
  let src;
  inputImagem.addEventListener('change', function (e) {
    const img = document.createElement('img');
    img.src = URL.createObjectURL(e.target.files[0]);
    img.onload = () => {
      src = cv.imread(img);
      cv.imshow("canvasOriginal", src);
    };
  });
  btnCinza.addEventListener('click', function () {
    let cinza = new cv.Mat();
    cv.cvtColor(src, cinza, cv.COLOR_RGBA2GRAY);
    cv.imshow("canvasSaida", cinza);
    cinza.delete();
  });
};
```

Original x escala de cinza no navegador



`canvasOriginal - cv.imread()`

`cv.cvtColor()`



RGBA → escala de cinza



`canvasSaida - cv.imshow()`

ATENÇÃO

Erros comuns nesta primeira aula



"cv is not defined"

Você tentou usar o OpenCV antes do WebAssembly terminar de carregar. Sempre escreva seu código dentro de `cv['onRuntimeInitialized']`.



Canvas em branco

Confira se o id do `<canvas>` no HTML é exatamente igual ao usado em `cv.imshow("id", ...)`.



Memory leak no navegador

Todo `cv.Mat()` criado manualmente precisa ser liberado com `.delete()` quando não for mais usado.



Live Server obrigatório

Abrir o `index.html` direto (`file:///`) pode bloquear a leitura de imagens — use sempre um servidor local.

Agora é a sua vez

- 1 Crie a pasta do projeto com a estrutura mostrada anteriormente.
- 2 Copie o HTML e importe o OpenCV.js pelo CDN.
- 3 Implemente o input de arquivo e os dois <canvas>.
- 4 No script.js, carregue a imagem escolhida com `cv.imread()`.
- 5 Adicione um botão que aplica `cv.cvtColor()` para escala de cinza.
- 6 Bônus: adicione um segundo botão com `cv.Canny()` para detectar bordas.

Desafio extra (opcional, vale ponto extra)

Desafio: Filtro de cores em tempo real via webcam

Usando `getUserMedia()` para acessar a webcam, capture frames em um `<canvas>` e aplique, com OpenCV.js, uma conversão de cor (ex.: `cv.COLOR_RGBA2GRAY` ou um limiar de cor) em tempo real, atualizando o resultado a cada frame com `requestAnimationFrame()`.

Entrega: envie o link do projeto (ou o .zip) pelo portal até a próxima aula.

O que vimos na Aula 1

- Diferença entre Processamento de Imagens e Visão Computacional.
- Como IA, Aprendizado de Máquina e Aprendizado Profundo se relacionam com VC.
- Aplicações reais: classificação, detecção, segmentação, reconhecimento facial e mais.
- Por que este curso é 100% prático, rodando no navegador.
- Ambiente configurado: editor, Live Server e OpenCV.js.
- Nosso primeiro programa: Hello Vision (ler imagem + converter para cinza).

Aula 2 · Fundamentos de Processamento de Imagens

Na próxima aula vamos abrir o "cv.Mat" por dentro: pixels, canais de cor (RGB), espaços de cor, histogramas e as primeiras operações ponto a ponto — sempre programando com OpenCV.js.

Traga para a próxima aula:

- Seu Hello Vision funcionando
- Dúvidas da aula de hoje
- Notebook com editor configurado



FIAP

Obrigado!

Visão Computacional Aplicada · Prof. Claudiney Sanches · profclaudiney.sanches@fiap.com.br

Material completo (apostila + código-fonte) disponível no portal da disciplina.